Week 3 →

Generalized policy iteration or GPI, Combine two parts; policy evaluation and improvement.

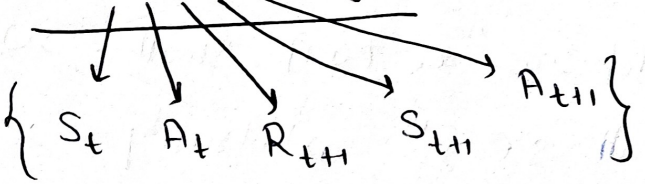
The first algorithm we saw with this form was policy iteration. Policy iteration runs policy evaluation to Convergence before gratifying the policy.

Then we saw GPI with Monte Carlo, which performs cycle of policy evaluation and improvement every episode. Here, GPI with MC does not perform a full policy evaluation step before improvement. Rather, it evaluates and improves after each episode.

Going even further, we could improve the policy after just one policy evaluation step (we will use this with TD).

To use TD within GPI, we need to learn the action value function.

The SARSA algorithm → (sarsa prediction)



Sarsa makes prediction about the values of state action pairs

Update eqn

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right)$$

(The algorithm we just specify is for policy evaluation, it learns action values for specific fixed policy.)

We can convert this into Control algorithm.
This time we will improve, the policy every time step rather than after an episode or after Convergence.

Sarsa → the GPI algorithm that uses TD for policy evaluation!

The Windy Gridworld →

Q-learning :→

Q-learning (off-policy TD Control) for estimating $\pi \approx \pi_*$

1) Algorithm parameters: Step size $\alpha \in (0,1]$, small $\epsilon > 0$
2) Initialize $Q(s,a)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$
3) Loop for each episode:
4) Initialize $\mathcal{S}$
5) Loop for each step of episode:

6) Choose A from S using derived from Q (e.g., ϵ-greedy)

7) Take action A, observe R, S'

8) $Q(s,A) \leftarrow Q(s,A) + \alpha [R + \gamma . \max_a Q(s',a) - Q(s,a)]$

9) $S \leftarrow S'$

10) Until S is terminal.

⇒ Why does Q learning 'max' instead of next state action pair ?

SARSA : $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma . Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$

$$q_\pi(s,a) = \sum_{s',a} p(s',a|s,a) \left(a + \gamma . \sum_{a'} \pi(a'|s') . q_\pi(s',a') \right)$$

Similar to bellman equations for action values!

SARSA is the sample based algorithm to solve the bellman equations for action values.

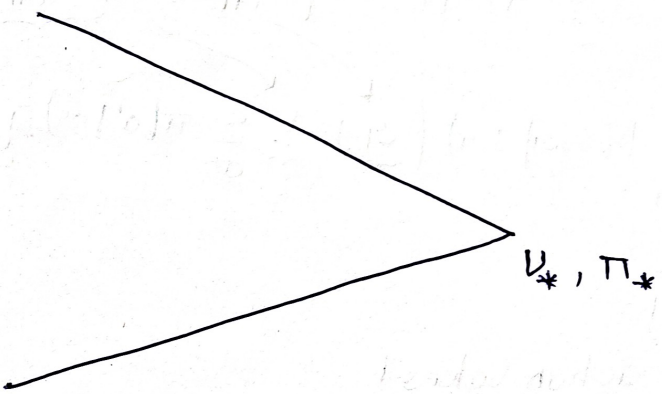
Q-learning also solves the bellman equations using the samples from the environment, but instead of using the standard bellman eqn, Q-learning uses the bellman's optimality eqn.

Q learning → $Q(S_t, A_t) \leftarrow \alpha \left[R_{t+1} + \gamma \max_a (S_{t+1}, a') - Q(S_t, A_t) \right]$

$$q_*(s,a) = \sum_{s',r} p(s',r|s,a) \left(r + \gamma \max_{a'} q_\pi(s',a') \right)$$

The optimality equations enable Q-learning to directly learn $\{q_*\}$ instead of switching b/w policy improvement and policy evaluation. steps.

∅ Connection to dynamic Programming :-



Sample based version of

Sarsa is ~~Knowledge~~ on policy iteration which uses bellman equations for action values that each depend on fixed policy.

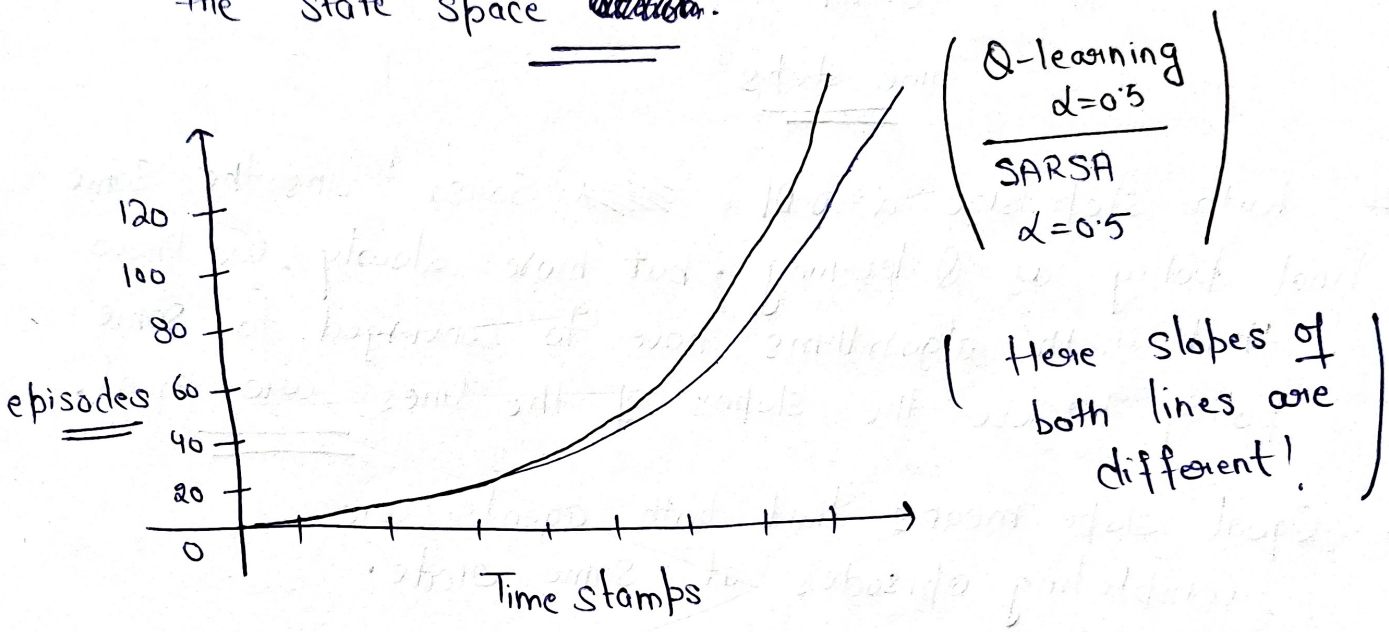
Q-learning is Sample based version of value iteration which iteratively applies bellman optimality eqn.

Applying the bellman optimality eqn strictly improves the value function, unless it is already optimal.

So value iteration continually improves as value

function estimate, which eventually converges to the optimal solution.

For the same reason, Q learning also converges to the optimal value function as long as the agent continues to explore and samples all areas of the state action space.



(Here slopes of both lines are different!)

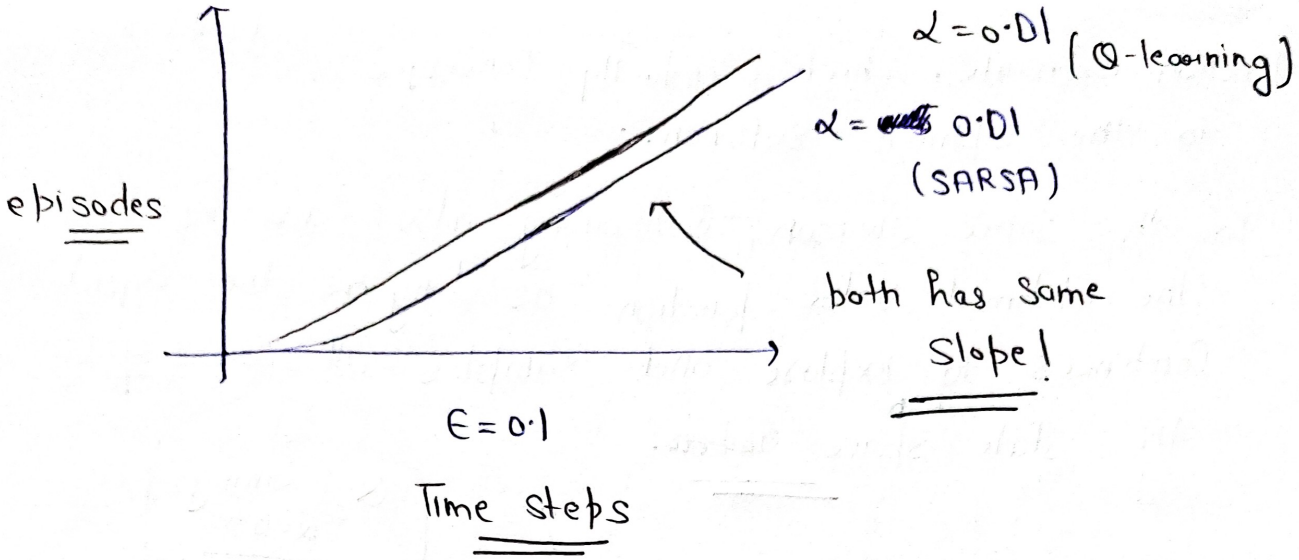
$\epsilon = 0.1$

→ Why Q-learning is better?

Q-learning takes the max over next action values. So it only changes when the agent learns that one action is better than another.

In contrast, SARSA uses the estimate of the next action value in its target. This changes every time the agent takes the exploratory action.

With step size $\alpha = 0.5$ (large) and taking exploratory action for the next state, SARSA is not able to learn optimal policy.



With step size $\alpha = 0.01$, Sarsa learns the same final policy as Q learning, but more slowly. We know that both algorithms have to converged to same policy because the slopes of the lines are equal.

Equal slope means that both agents are completing episodes at same rate.

How is Q learning off policy? →

(without using importance sampling)

So far we have seen off policy algorithms that uses important sampling.

Sarsa: $$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left(R_{t+1} + \gamma . Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right)$$

#

agent estimates its value function according to expected returns under their target policy. They actually behave according to their behaviour policy.

When the target policy and behaviour policy are the same, the agent is learning on-policy, otherwise, the agent is learning off-policy.

Sarsa is an on-policy algorithm. In Sarsa, the agent bootstraps off of the value its going to take next, which is sampled from its behaviour policy.

Q-learning instead bootstraps off of the largest action value in its next state. This is like sampling an action under an estimate of the optimal policy rather than behaviour policy.

Since the Q-learning learns about the best action it could possibly take rather than actions it actually takes, it is learning off policy.

➡ What are the targets and behaviour policy!

Q-learning's target policy is always greedy with respect to its current values. However its behaviour policy can be anything!

One possible behaviour policy is epsilon greedy!

The difference b/w target and behaviour policies confirms that Q-learning is off-policy.

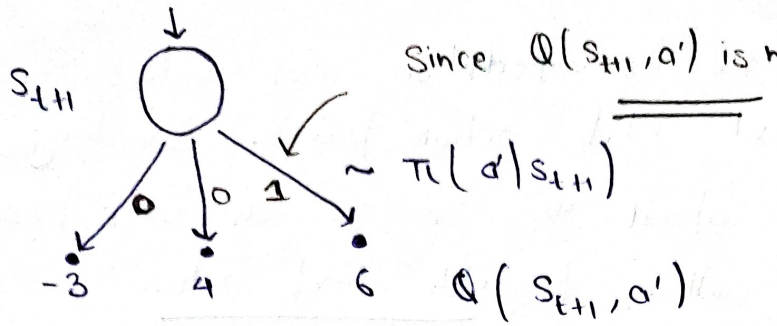
→ But why here we are not seeing any importance sampling ratios?

It is because the agent is estimating action values with unknown policy. It does not need important sampling ratios to correct for the difference in action selection.

The action value function represents the returns following each action in given state. The agents target policy represents the probability of taking each action in a given state.

Putting these two elements together, the agent can calculate the expected return under its target policy from any given state, in particular S_{t+1}.

Q-learning uses exactly this technique to learn off-policy. Since the agent's target policies is greedy, with respect to its action values, all non-maximum actions have probability 0.



$$\sum_{a'} \pi(a'|S_{t+1})\, Q(S_{t+1}, a') = \mathbb{E}_{\pi}[G_{t+1} | S_{t+1}]$$

As a result, the expected return from that state is equal to the maximal action value from that state.

We know that Q-learning doesn't iterate blw policy evaluation and improvement, but rather learns the optimal values directly. Directly learning the optimal value function is great but there are some disadvantages that make less desirable in specific situations.

e.g Cliff Walking environment.

Expected Sarsa →

Bellman eqn for acth values :

$$q_{\pi}(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s')\, q_{\pi}(s',a') \right]$$

$\sim p(s',r|s,a)$

$\sim \pi(a'|s')$

SARSA : $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right)$

Here we can see the expectation over values of possible next state action pairs. Breaking this expectation apart we see a sum over possible next states as well as possible next action.

Sarsa estimates this expectation by sampling the next state from the environment and the next action from its policy. $\longrightarrow \sim \pi(a'|s')$

However, the agent already know the policy $(\pi(a'|s'))$, so why should it (have to) sample its next action?

Instead, it should just compute the expectation directly. In this case we can take weighted sum of the values of all possible next actions. The weights are the probability of taking each action under the agents policy.

Explicity computing the expectations over next actions is the main idea behind the expected SARSA algorithm.

$$\begin{cases} \gamma \cdot Q(S_{t+1}, A_{t+1}) \\ \uparrow \\ \gamma \cdot \sum_{a'} \pi(a'|S_{t+1}) \cdot Q(S_{t+1}, a') \end{cases}$$

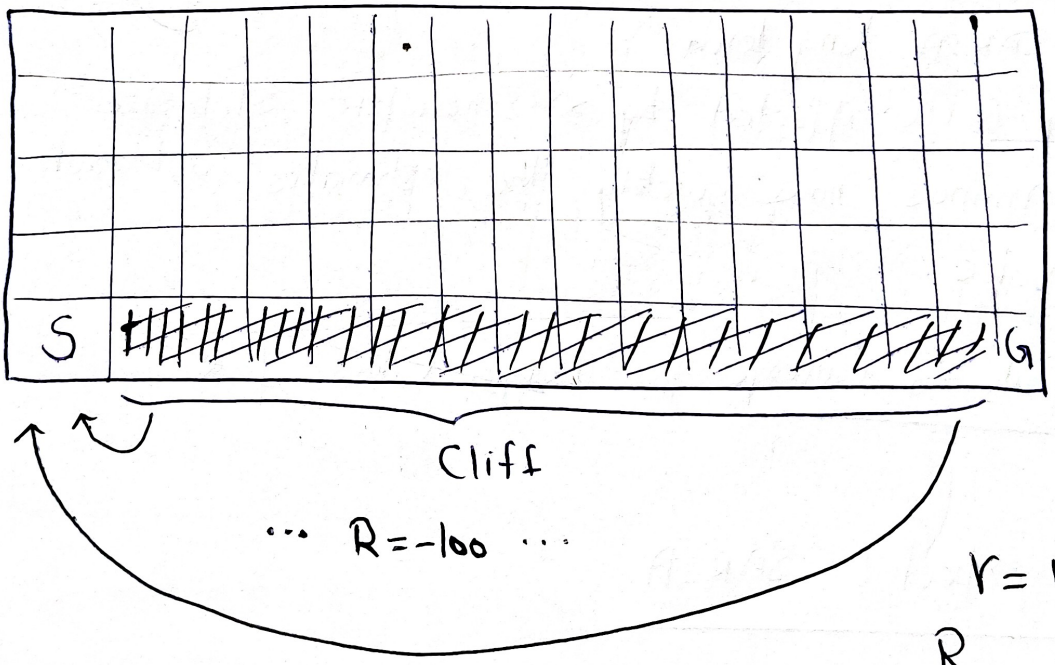
Expected SARSA:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left(R_{t+1} + \gamma \sum_{a'} \pi(a'|S_{t+1}) \cdot Q(S_{t+1}, a') - Q(S_t, A_t)\right)$$

The expected SARSA is identical to the SARSA except, the T-error uses the expected estimate of the next action value instead of a sample of the next action value.

Expected SARSA has more stable update target than SARSA. In general, expected SARSA's update targets are much lower variance than SARSA's.

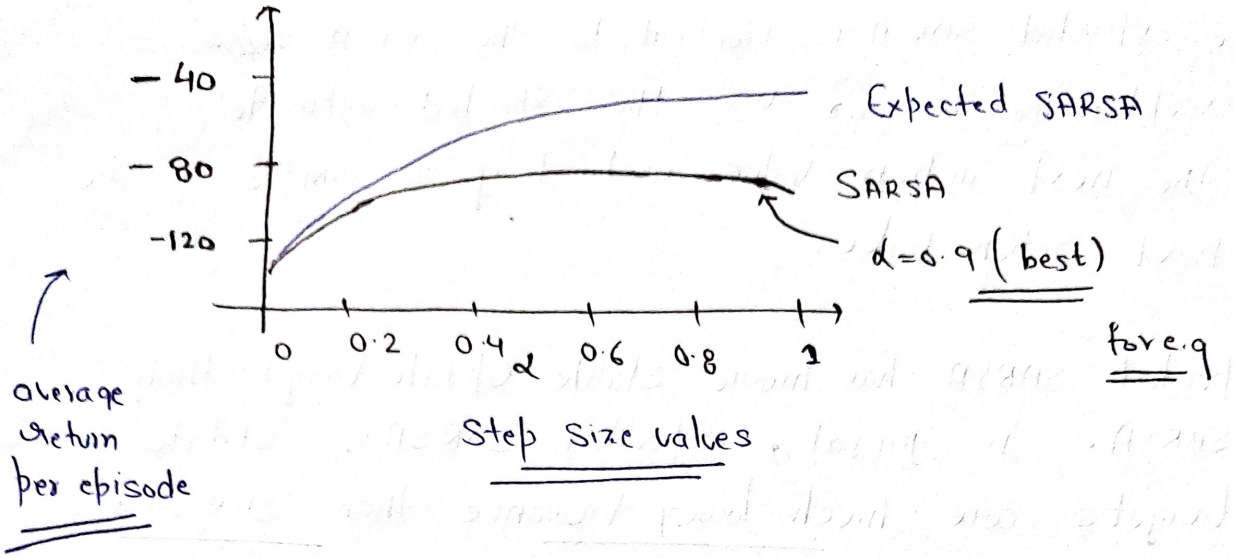
The lower variance comes with a downside though. Computing the average over next actions becomes more expensive as the number of action increases.



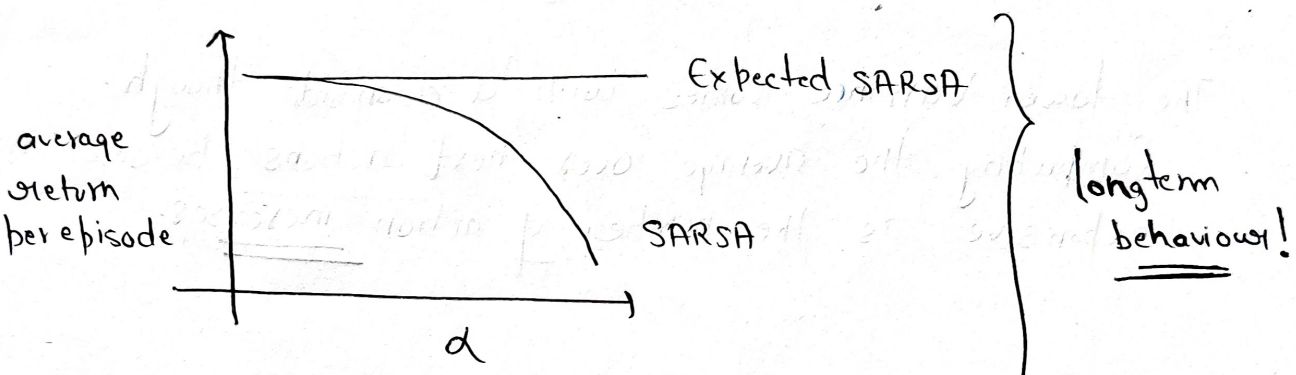
$r = 1$

$R_{step} = -1$

SARSA performed better than Q-learning on this domain because SARSA's policy accounted for its own exploration.



for e.g



Expected SARSA long term behaviour is unaffected by α. Therefore Step size only determines how quickly the estimates approach their target value.

SARSA fails to Converge for larger values of α

OFF policy expected SARSA

ON policy :

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left(R_{t+1} + \gamma \sum_{a'} \pi(a'|S_{t+1}) \cdot Q(S_{t+1}, a') - Q(S_t, A_t)\right)$$

{ ⇒ here behaviour policy and target policy are equal }

Here $A_{t+1} \sim \pi(a'|S_{t+1})$

In expected SARSA, next action is sampled from 'π'. However expectation over actions is computed independently of the action actually selected in the next state. In fact π need not be equal to the behaviour policy.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left(R_{t+1} + \gamma \underbrace{\sum_{a'} \pi(a'|S_{t+1}) Q(S_{t+1}, a')}_{\uparrow} - Q(S_t, A_t)\right)$$

$$\cancel{A_{t+1}} \sim \pi(a'|S_{t+1}) \neq b(a'|S_{t+1})$$

$$\downarrow$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left(R_{t+1} + \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\right)$$

This means expected SARSA, like Q-learning can be used to learn off-policy without importance sampling.

Greedy Expected SARSA →

Now what happens if the target policy is greedy?

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left(R_{t+1} + \gamma \sum_{a'} \pi(a'|S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t)\right)$$

$\pi = \pi^*$?

Thus Q-learning is a special case of expected SARSA!

Expected SARSA uses the same technique as Q-learning to learn off policy without importance Sampling.